



NAME :- SOUMYA VERMA

BRANCH :- AI/ML

REGISTRATION NO. :- 25BAI11204

YEAR :- 2nd` YEAR

COURSE CODE :- CSE2006

DATE :- 17-09-2026

**Topic :- BILLING & INVENTORY
MANAGEMENT SYSTEM**

1. Introduction

In standard retail electronics stores, managing product stock, calculating itemized taxes, and handling customer billing manually often causes long queues, calculation mistakes, and messy record-keeping. Electronics items like Keyboards, Monitors, and Accessories require continuous stock tracking along with standard 18% GST and store discounts.

To solve this, I designed and developed the **Electronics Billing & Inventory System** in Java. It provides a simple desktop-based Graphical User Interface (GUI) built using Java Swing. It connects directly with a MySQL database via JDBC to manage inventory stock, process quick customer checkouts with a 10% store-wide discount, apply 18% GST, and store customer transaction history permanently.

2. Problem Statement

Small to mid-sized electronics stores encounter several daily operational issues:

- Difficulty in keeping accurate track of current stock, leading to accidental overselling or stock shortages.
- Time-consuming manual calculations for standard 18% GST tax rates and flat promotional discounts (10%).
- Loss of past billing records due to lack of an organized database to view or remove old transaction logs.

Proposed Solution: A desktop application that automates real-time stock management, applies a flat 10% discount and 18% GST automatically, generates printable checkout receipts, and saves customer order records into MySQL.

3. Functional Requirements

The system contains four major core modules:

- **Inventory CRUD Operations:** Complete capability to Add new electronics products, Read stock items, Update prices/quantities, and Delete products using a custom "Manage Products" dialog window.
- **Automatic Stock Refill Logic:** Re-adding an existing product name automatically increments the available stock count instead of duplicating database records.
- **Smart Cart & Billing Engine:** Inputs customer details (Name and Phone) and automatically calculates a flat **10% Auto-Discount** on item price, adds **18% GST**, and generates a formal receipt output.
- **Order History & Deletion Management:** Permanent storage of orders in MySQL, along with an interactive pop-up table to view past sales and delete outdated customer orders.

4. Non-Functional Requirements

The system addresses the following key quality attributes:

- **Performance:** Very fast database response times (under 100ms) for inserting and fetching items via JDBC connections.
- **Usability:** A single split-screen dashboard layout designed with intuitive Swing controls for fast operational workflow.
- **Reliability:** Primary key constraints on product_id and order_id prevent duplicate entries and maintain database integrity.
- **Error Handling & Validation:** Built-in validation checks (using JOptionPane) to handle negative quantity inputs, missing fields, or empty cart errors cleanly.

5. System Architecture

The application uses a 3-Tier Layered Architecture:

[Presentation Layer] ---> Java Swing Desktop Interface (BillingUI.java)

|

[Business / DAO Layer] ---> Data Access Classes (ProductDAO.java, OrderDAO.java)

|

[Data Storage Layer] ---> MySQL Database Server (billing_inventory_db)

6. Design Diagrams

Use Case Diagram

- **Actor:** Store Manager / Billing Cashier
- **Core Actions:** Add Product to Inventory, Delete/Manage Stock, Add Items to Cart, Generate Bill Receipt, View Past Customer Orders, Delete Order History.

Database Schema / ER Diagram

- **products Table:** product_id (PK, Auto-Increment), product_name (VARCHAR), price (DOUBLE), quantity (INT).
- **orders Table:** order_id (PK, Auto-Increment), customer_name (VARCHAR), customer_phone (VARCHAR), total_amount (DOUBLE), order_date (TIMESTAMP).

7. Design Decisions & Rationale

- **Java Swing Framework:** Chosen because it comes built-in with Java, requires no external dependencies, and creates efficient cross-platform desktop UI components.
- **JDBC PreparedStatement:** Implemented across all DAO queries to protect against SQL injection and optimize database query performance.
- **Modular DAO Structure:** Keeps UI logic completely separate from database query code, making future codebase updates clean and easy to manage.

8. Implementation Details

The project source code is organized into clean, modular packages:

- database.DBConnection: Establishes connections with the MySQL billing_inventory_db database.
- model.Product & model.Order: Encapsulates data fields, getters, setters, and constructors.
- dao.ProductDAO & dao.OrderDAO: Handles database queries (INSERT, SELECT, UPDATE, DELETE).
- BillingUI.java: Main GUI screen containing form inputs, cart table, totals summary, receipt display panel, and event handlers.

9. Screenshots / System Output

Generated Receipt				
=====				
INVENTORY BILLING RECEIPT				
=====				
Customer Name : Tanishqa				
Phone Number : 9874561230				

Item	Price	Qty	Disc	Total

Laptop	50000.0	1	10.0%	45000.00
Keyboard	3000.0	1	10.0%	2700.00
Mouse	2500.0	1	10.0%	2250.00

Sub Total : ₹49950.00				
GST (18%) : ₹8991.00				
Grand Total : ₹58941.00				
=====				
Thank You For Shopping!				

ID	Product Name	Price (₹)	Stock
13	CPU	20000.0	80
14	Monitor	45000.0	75
15	Keyboard	3000.0	100
16	Mouse	2500.0	150
17	Laptop	50000.0	100
18	Printer	25000.0	110
19	Hard Disc	14000.0	85
20	Pendrive	200.0	200
21	RAM	12900.0	25
22	X-Box	200000.0	10
23	Iphone	300000.0	5
24	Samsung	10000.0	15

Delete Selected Product

Order ID	Customer Name	Phone	Total Amount (₹)
9	Sita	7049879256	₹48462.6
8	Sam	6254310089	₹53100.0
6	Tanishqa	9874561230	₹58941.0
5	Rishya	8889074529	₹106200.0

Delete Selected Order

Navigation
 Filter objects
 billing_inventory_db
 Tables
 Views
 Stored Procedures
 Functions
 billing_system
 Tables
 bill_items
 bills
 customers
 products
 users
 Views
 Stored Procedures
 Functions
 placement_system
 sakila
 sys
 world

Limit to 1000 rows

```

7      price DOUBLE NOT NULL,
8      quantity INT NOT NULL
9    })
10
11  * USE billing_inventory_db;
12  * SELECT * FROM products;
13
14  * USE billing_inventory_db;
15
16  * CREATE TABLE orders (
17    order_id INT AUTO_INCREMENT PRIMARY KEY,
18    customer_name VARCHAR(100) NOT NULL,
19    customer_phone VARCHAR(15) NOT NULL,
20    total_amount DOUBLE NOT NULL,
21    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
22  )

```

Output
 Action Output

#	Time	Action	Message
1	19:34:40	CREATE DATABASE billing_inventory_db	1 row(s) affected
2	19:34:40	USE billing_inventory_db	0 row(s) affected
3	19:34:40	CREATE TABLE products (product_id INT AUTO_INCREMENT PRIMARY KEY, product_name VARCHAR(100) NOT NULL, price DOUBLE NOT NULL, quantity INT NOT NULL)	0 row(s) affected
4	19:50:08	USE billing_inventory_db	0 row(s) affected
5	19:50:08	SELECT * FROM products LIMIT 0, 1000	1 row(s) returned
6	20:14:30	CREATE TABLE orders (order_id INT AUTO_INCREMENT PRIMARY KEY, customer_name VARCHAR(100) NOT NULL, customer_phone VARCHAR(15) NOT NULL, total_amount DOUBLE NOT NULL, order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP)	0 row(s) affected

- Main Billing Interface:** Showing Customer Form, Cart Table with 10% Discount, 18% GST summary, and generated receipt.
- Manage Inventory Dialog:** Showing the product table with the "Delete Selected Product" functionality.
- Past Order History Dialog:** Showing saved customer billing records with the "Delete Selected Order" button.

10. Testing Approach

- **Unit & DAO Testing:** Tested database methods (addProduct, deleteProduct, saveOrder, deleteOrder) individually to ensure database changes reflected correctly.
- **Input Validation Testing:** Verified that entering blank customer names or non-numeric stock values triggers proper warning dialogs.
- **End-to-End Integration Testing:** Confirmed that generating a bill accurately updates inventory dropdowns, calculates exact 10% discount and 18% GST values, and saves the final invoice in MySQL.

11. Challenges Faced & Solutions

- **Issue 1 (Duplicate Stock):** Adding existing products created duplicate database rows.
 - *Solution:* Modified ProductDAO to query existing names first and update the stock quantity if the item already exists.
- **Issue 2 (Method Resolution Error):** Enriched Order.java with a missing setOrderId() setter method to ensure primary keys pass correctly during past order deletion operations.

12. Learnings & Key Takeaways

- Practical experience in integrating Java Swing desktop GUIs with MySQL databases through JDBC.
- Hands-on knowledge of implementing Data Access Object (DAO) patterns and modular software design.
- Real-world understanding of managing commercial calculations like percentage discounts and GST within transactional software.

13. Future Enhancements

- Option to export or print generated billing receipts directly into PDF files.
- Multi-user login system separating Admin privileges (stock control) from Cashier capabilities (billing only).
- Barcode scanner integration for automated product identification during checkout.

14. References

1. Oracle Java Swing Official Guide: <https://docs.oracle.com/javase/tutorial/uiswing/>
2. MySQL Connector/J Documentation: <https://dev.mysql.com/doc/connector-j/8.0/en/>
3. Java Database Connectivity (JDBC) API Documentation: <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>